

# ContextNET - Contextual classification of building components using deep neural networks

Wojciech Teclaw<sup>a,b</sup>, Artur Tomczak<sup>b</sup>, Mateusz Kasznia<sup>c</sup>, Marcin Luczkowski<sup>d</sup>,  
Nathalie Labonnote<sup>a</sup>, Eilif Hjelseth<sup>b</sup>

<sup>a</sup> SINTEF Community, Høgskoleringen 7B, 7034, Trondheim, Norway

<sup>b</sup> Norwegian University of Science and Technology, Høgskoleringen 7A, 7034, Trondheim, Norway

<sup>c</sup> Make IT, Brzozowa 2, 32085, Modlniczka, Malopolska, Poland

<sup>d</sup> Norwegian University of Science and Technology, Richard Birkelands vei 1A, 7034, Trondheim, Norway

## ARTICLE INFO

### Keywords:

IFC  
Building information modelling  
Deep learning  
Building component classification  
Machine learning

## ABSTRACT

In recent decades, the construction industry has made significant steps towards digitalisation. One of the most persistent challenges remains ensuring data accuracy, particularly in building component classification. Recent research has explored the application of machine learning algorithms for element classification within Building Information Modelling (BIM). These methods, while effective in handling individual geometric features, often overlook the critical influence of an element's spatial context, leading to misclassifications in dense and interconnected models. To address this gap, ContextNet proposes an innovative deep-learning approach that integrates contextual information from surrounding components, enhancing classification accuracy by accounting for spatial relationships. By processing point clouds from both individual elements and their context, ContextNet enables more robust identification of building components within BIM models. Results demonstrate that ContextNet significantly improves classification accuracy and robustness, reducing common misclassifications in complex BIM datasets. The proposed framework enhances BIM data by enabling context-aware component recognition, supporting more accurate and dependable classification within the construction industry.

## 1. Introduction

Over the decades, the architecture, engineering, and construction (AEC) industry has undergone significant transformations due to advancements in digital technologies. The first major shift was the replacement of manual drawing boards with computer-aided design (CAD) [1]. This transition not only accelerated daily tasks but also enabled more efficient revisions and project modifications. Simultaneously, seeking to enhance not just human interpretation but also computer processing capabilities in design and construction workflows [2].

Building Information Modelling (BIM), originally conceptualised by Eastman, revolutionised the industry by integrating virtual building models with comprehensive databases, advanced visualisation tools, and quantitative analysis methods [3,4]. This innovation addressed the increasing complexity of construction projects. An important milestone in BIM's popularisation was the formation of the International Alliance for Interoperability (IAI) in 1994, nowadays known as buildingSMART [2]. The subsequent release of the Industry Foundation

Classes (IFC) in 1997 further promoted industry-wide interoperability, standardisation, and productivity improvements [2]. Now, after more than two decades of continuous development, the IFC is a dominant BIM data exchange standard. It is also an integral component of BIM methodology supported by the ISO 19650 standard about information management across the entire lifecycle of construction projects [5]. Its role in facilitating effective data exchange has positioned it as a critical element in modern construction processes [6,7].

Consequently, IFC has gained widespread recognition among stakeholders for its feasibility and efficacy, not only during the design phase but as a dynamic component that evolves alongside the construction object [8]. Throughout the construction process, building information models are progressively enriched with detailed data, such as material specifications, construction methodologies, and as-built conditions [9–11]. However, a significant challenge associated with these models is ensuring their reliability and validity as trustworthy sources of information [12,13].

To address this challenge, various research initiatives have been proposed, ranging from manual model reviews, rule-based compliance

\* Corresponding author at: SINTEF Community, Høgskoleringen 7B, 7034, Trondheim, Norway.  
E-mail address: [wojciech.teclaw@sintef.no](mailto:wojciech.teclaw@sintef.no) (W. Teclaw).

checking [14], semi-automated validation of component relations [15] and, more recently, fully automated workflows using artificial intelligence (AI) [16]. AI has gained recognition for its ability to streamline validation processes by automating the extraction of stand-alone components, converting them into a chosen data structure, and classifying them into predefined categories through neural networks. This approach offers scalability, reduces the potential for human error in model validation and is significantly faster than a manual process.

However, while widely adopted, AI-based classification methods have their limitations, including a predominant focus on the geometry of individual elements. A component's surrounding environment, including its spatial relationships and interaction with other elements, often plays a crucial role in accurately identifying its function. For example, distinguishing the difference between a column and a pipe depends not only on the element's geometry but also on its location, shape and surroundings. By ignoring the geometrical context, stand-alone component workflows risk misclassifications, especially in complex models with highly interdependent elements.

This study aims to enhance the accuracy and reliability of element classification by proposing a more efficient and robust machine learning model called ContextNet that analyses components in relation to their surrounding elements using Point Cloud Data. The study is structured around two main research questions (RQ):

1. How to integrate an element and its surrounding information in object classification tasks? (RQ1)
2. How does incorporating contextual information affect the accuracy of building component classification? (RQ2)

The paper begins with an Introduction in Section 1, which presents the theoretical foundation and defines the research problem. Section 2 provides a review of relevant literature, existing methodologies, and notable case studies. In Section 3, the machine learning components and models used for dataset preparation are described. Section 4 outlines the dataset preparation process, including the extraction of contextual information, data normalisation, augmentation, and dataset generation rules. This section also details the architecture of the ContextNet model, including its loss function and training specifics.

Section 5 presents the results. Section 6 offers a thorough analysis of the results, comparing the performance of two neural networks and examining their implications for real-world construction projects. This section also discusses limitations, data processing challenges, and model efficiency. Finally, the conclusion summarises key findings, highlights their broader significance, and suggests directions for future research.

## 2. Background

### 2.1. Problem of misclassified building models

Building information models rarely align with normative terms, and the resulting mismatch complicates the compliance checking [17–19]. Misclassification can sometimes occur due to differences in the software used for authoring [20]. Although BIM tools apply default classifications by mapping internal element categories to normative standards, these mappings are often inconsistent and error-prone, largely due to variations in user practices and algorithmic interpretations—highlighting the need for advanced methods. The accurate classification of building components within a building information model is crucial for ensuring data integrity and enabling efficient multidisciplinary design, cost estimation, quantity take-off, bidding, building permits, simulations, analysis and construction planning [10,17,20,21]. In contrast, poor semantic annotation can result in incorrect assessments of potentially serious consequences like increased costs, delays, reduced efficiency and safety [17,21]. Moreover, inconsistencies in object classification can hinder interoperability between BIM platforms and analysis

tools, impeding workflows and requiring manual corrections. [22] estimated the cost of low industry interoperability, taking into account related avoidance, mitigation and delay costs, to be between 1%–2% of industry revenue in the US.

This challenge is being addressed by semantic enrichment, which supplements the data with inferred semantics. The literature review by [23] lists the main contexts of application of the semantic enrichment to be capturing existing [building] conditions, validating code compliance, authoring design and 4D models, reviews, site selection and logistics, design coordination, energy performance analysis, and others, such as integration with GIS (Geographic Information System). Recent studies indicate that machine learning models are among the most widely used methods for validating model components, enabling automated component classification, reducing manual workload, and simplifying the workflow [10].

### 2.2. Machine learning models for class prediction

Several efforts have been made to train machine learning (ML) models using only numerical features describing the geometry of building elements. For example, Krijnen and Tamke [24] utilised geometric properties, such as area, volume, and radius of gyration, to identify patterns, enhance classification, and flag inconsistencies within models. Their work highlights the potential of machine learning to uncover implicit knowledge within BIM models, which is often overlooked in traditional methods focusing on explicit data. Similarly, Wang et al. [25] incorporated location, mesh parameters, and dimensions to perform semantic enrichment and infer relationships between elements through a graph-based approach, deploying eight different machine learning algorithms. Weibach-Eyüboğlu et al. [21] further extended this approach by basing their AI model on features like geometrical dimensions, spatial relationships, and geometric complexity. Both studies underscore the effectiveness of ML in classifying repetitive components with low variance, although they require extensive and varied datasets to perform well.

Advanced neural network models have also shown promising results. Emunds et al. [26] conducted a comparative study using a benchmark dataset, finding that Multi-View Convolutional Neural Networks (MVCNN) yielded superior results compared to other models, such as MeshNet Feng et al. [27] and the Dynamic Graph Convolutional Neural Network (DGCNN) by Wang Wang et al. [25]. Notably, the DGCNN recalculates local neighbourhood relationships dynamically at each layer, allowing it to capture both local and non-local features within point clouds. The core EdgeConv operation in DGCNN constructs edge features between neighbouring points, enabling the model to learn spatial structures and semantic relationships effectively.

Additionally, models like MeshNet have been specifically designed to process 3D shape representations by learning directly from mesh data, which includes vertices, edges, and faces. MeshNet addresses the challenges of mesh data's inherent complexity and irregularity by processing each face as an independent unit and splitting features into spatial and structural descriptors, aggregating them through mesh convolutions. This approach enables MeshNet to capture detailed geometric patterns and has shown competitive performance on tasks like 3D shape classification and retrieval Feng et al. [27].

In the field of point cloud data, models such as PointNet (Qi et al. [28]) have been effective in capturing 3D geometric structures. PointNet processes raw point clouds without voxelisation or rendering, enabling direct application to classification, part segmentation, and scene parsing tasks. Similarly, IfcVoxNet, proposed by Luttun and Krijnen [29], uses a voxel-based approach to handle segmentation and classification of IFC building models by incorporating the spatial context surrounding elements, enabling the model to recognise building elements regardless of modelling variations, such as wall joints or window details. While IfcVoxNet demonstrated accuracy in classifying

major IFC classes, its scope was limited due to the small dataset of only 10 IFC models, which under-represented certain classes.

Probabilistic methods like Random Forest, Naive Bayes, and CatBoost were evaluated by Petrochenko et al. [30] for IFC-based BIM classification. Among these, CatBoost achieved the highest accuracy and F1-score, demonstrating strong reliability in handling complex data for BIM element classification. Zhai et al. [31] addressed the shortage of labelled data in BIM by developing a BIM-driven data augmentation method that synthesises annotated point clouds for deep learning in semantic segmentation tasks. Their approach used synthetic point clouds to enhance training datasets, proving effective in improving segmentation accuracy in large indoor models.

Various ML models were employed to address semantic integrity in BIM models, including the Support Vector Machine (SVM) used by Koo et al. [32]. Trained on geometric and relational BIM data, the SVM distinguished between IFC classes with high precision, proving valuable for automated quality checks in BIM deliverables by ensuring consistent IFC mappings. Rogage and Doukari [33] used Convolutional Deep Belief Networks (CDBN) to classify 3D objects within BIM models, aiming to automate semantic data enrichment and maintain data quality throughout the building lifecycle.

Finally, graph-based approaches have been explored for BIM classification. Collins et al. [34] presented a Graph Convolutional Network (GCN) to classify IFC elements by extracting features from triangle meshes through geometric deep learning. Although graphs are effective in representing individual objects, they have also been employed to capture the relationships between components, thereby enhancing semantic enrichment. Graph-based methods have shown particular efficiency in the room type classification of BIM models. For example, Wang et al. [35] proposed the SAGE-E architecture — a variant of the GraphSAGE model — that leverages graph relations for improved classification. In this framework, room entities are modelled as nodes in a graph, with edges representing spatial relationships such as adjacency or shared boundaries. The SAGE-E model integrates node features (including geometric properties such as area, perimeter and shape) with edge features that represent the adjacency between rooms categorised into three groups. The use of GCN for room classification was also investigated by Buruzs et al. [36], which highlights the importance of contextual information. An alternative approach for incorporating context was demonstrated by Bloch and Sacks [18], who combined rule-based inferencing with machine learning techniques to enhance semantic enrichment in BIM models. Their study addressed the limitations of conventional methods in regulatory compliance by showing that data-driven approaches can provide more reliable semantic labels despite variable modelling practices. Furthermore, Austern et al. [37] examined the usage of GNN for BIM classification by constructing proximity graphs in which building elements are represented as nodes and edges that capture spatial relationships based on geometric intersections. Their results indicate that integrating contextual relationships improves classification accuracy. The explicit encoding of neighbouring element categories can further boost accuracy under ideal conditions, but alternative classifiers that do not rely on such encoding are also capable of reaching a high accuracy. This finding suggests that the overall benefits of incorporating contextual information are maintained even when labelling errors are present, which underscores the value of evaluating multiple classifier variants.

A majority of the reviewed methods primarily focus on individual element geometry. Many studies have successfully applied machine learning techniques to leverage geometric properties, but these methods often overlook the spatial context surrounding elements. Incorporating information about neighbouring components in the room type enrichment process [35,36] and the classification task [29,37] can enhance performance by capturing spatial relationships among elements. However, this area offers significant potential for further development. The studies by Austern et al. [37] and Luttun and Krijnen [29] operate on a limited number of classes (10 and 9, respectively)

and primarily focus on components in the architectural and structural domains. Therefore, there is a need to develop neural network models that are capable of achieving higher accuracy across a larger number of classes. Moreover, while Austern et al. [37] focuses on graph neural networks and Luttun and Krijnen [29] uses voxel-based methods, there remains a lack of efficient methods for incorporating context using point cloud representations in the AECO domain. One of point clouds' advantages is their flexibility when it comes to sampling range, number of points per element, which makes it a flexible entry point for context incorporation and improving the general classification accuracy of BIM components.

### 3. Materials

The following section outlines the materials utilised for developing, testing, and evaluating the proposed ContextNet neural network. It provides an overview of the BIM models used for data capture, followed by a detailed explanation of the Spatial Transformer Network and PointNet neural network architectures.

#### 3.1. IFC models dataset

The availability of realistic IFC models for neural network training remains limited, largely due to restrictions on data sharing tied to intellectual property rights and security concerns. Existing models are often simple academic examples, either lacking the necessary complexity and realism or being too detailed, unlike actual projects.

We found two datasets of individual elements with IFC labels: 884 thousand elements curated by [38] and another one with 19,613 confirmed objects distributed over 65 classes from [39]. Those collections provide substantial help in training individual geometry recognition. However, they are not ideal for this project due to the lack of surrounding context.

Instead, we sought publicly available IFC models to extract element geometry and context. Out of 245 IFC models found, 160 are sourced from [40]. Additionally, 55 models were extracted from [41], including multiple discipline files from the S1 project donated by Stam + De Koning construction company. Other contributions include four high-quality discipline models from RWTH Aachen University [42], six from BimWhale [43], another six from the [44], five from the [45], five from Steptools.com, and the remaining four from the [46]. The resultant dataset contains samples of 84 IFC entities, with the predominant being linear elements like structural beams, columns, members, reinforcement, pipes and ducts.

#### 3.2. Spatial transformer network

The Spatial Transformer Network (TNet) is a neural network module designed by [47] to address common orientation and position variations in 2D data. It learns to apply affine transformations, aligning each object in the data to a consistent canonical form. This alignment minimises spatial discrepancies across samples, which is crucial for handling data that may come in various orientations and enhances the network's ability to focus on shape and structural features. Moreover, this module can also be applied in 3D environments, as demonstrated in architectures like PointNet, where it helps handle variations in 3D point cloud data effectively [28].

The TNet architecture, shown in Fig. 1, operates by predicting a transformation matrix that adjusts the input data. It receives an input feature tensor of dimensions  $B \times N \times F$ , where  $B$  is the batch size,  $N$  is the number of points, and  $F$  represents the feature dimensions (e.g.,  $x$ ,  $y$ , and  $z$  coordinates). The network processes this tensor through a series of shared multi-layer perceptrons (MLPs) with hidden units  $H$ , producing a transformation matrix of size  $F \times F$ . This matrix is then applied to the original features through a matrix multiplication step, aligning the data to a canonical orientation. An additional trainable

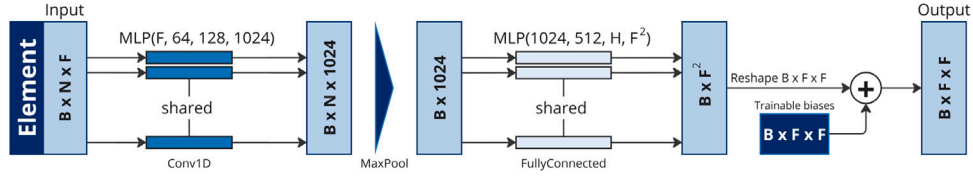


Fig. 1. Spatial Transformer Network (TNet) architecture, illustrating the transformation of input features  $B \times N \times F$  through shared MLPs to predict a transformation matrix of size  $F \times F$ . The output transformation matrix, combined with trainable biases, aligns data to a canonical form to mitigate spatial variability.

Source: Inspired by [47].

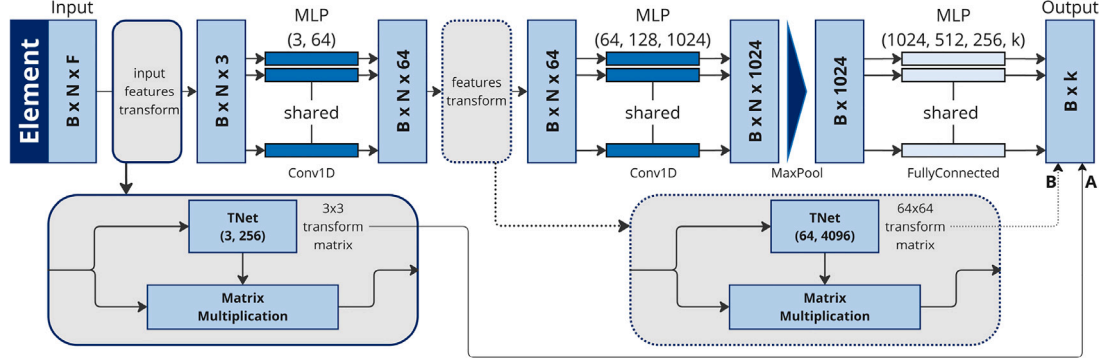


Fig. 2. PointNet architecture, illustrating input feature encoding with shared MLPs, global feature aggregation through max-pooling, and final classification with fully connected layers.

Source: Inspired by [28].

bias term enhances this transformation, allowing the network to capture more complex spatial variations. This specific adaptation of TNet for 3D data was originally proposed in the PointNet paper to handle transformations in point cloud data [28].

### 3.3. PointNet

The entry for building the proposed framework begins with PointNet architecture, designed by [28]. PointNet enables the classification and segmentation of point clouds based on spatial coordinates. The input to the PointNet model is a set of  $N$  points, each with  $F$  features, as shown in Fig. 2 at the NN entry point. For this application, the PointNet model uses three features representing the  $x$ ,  $y$ , and  $z$  coordinates.

The initial stage of PointNet involves transforming the input data using a Spatial Transformer Network (TNet), discussed in Section 3.2. The primary function of TNet is to align the input point cloud to a canonical orientation, thus reducing the variance due to rotational and translational discrepancies across data samples.

Following this alignment, each point is processed individually through a shared multi-layer perceptron (MLP), which encodes the point features into a higher-dimensional space. This step is crucial for capturing intricate local features while preserving each point's characteristics. A second TNet layer, referred to as the feature transformer, is then applied to refine the feature space, ensuring robust feature extraction and enhancing invariance to spatial transformations.

The transformed point features are then passed through additional MLP layers, after which a symmetric max-pooling function aggregates these features into a single global descriptor. This global descriptor serves as a comprehensive representation of the entire point cloud, encapsulating the overall shape and structure while maintaining permutation invariance.

Finally, the aggregated global features are fed through several fully connected layers using MLPs, producing a  $k$ -dimensional vector that represents the probability distribution across classes, thus providing the final classification output based on the learned features. Additionally, the two blue-dashed lines illustrate how transformation matrices are carried through to the output, enabling further loss calculations to enhance alignment accuracy.

In this framework, PointNet serves as a foundational comparison model to ContextNet, establishing a benchmark for validating the learning process across classification tasks. All performance metrics, including *accuracy*, *loss*, and computational efficiency, are measured to assess the effectiveness of incorporating context-based learning into the point cloud analysis. These metrics provide essential insight into model improvements, with *accuracy* indicating the percentage of correctly classified samples and *loss* reflecting the model's prediction precision relative to true labels. Additionally, PointNet's simplicity allows for baseline evaluations, clarifying how ContextNet's extended context features influence both learning dynamics and overall classification performance.

## 4. Methods

### 4.1. Data preparation

#### 4.1.1. BIM element extraction

The core element of the proposed methodology is a machine learning model, which depends on datasets used for training, testing, and validation. A review of publicly available datasets revealed that existing datasets for element classification ML models contain information about isolated BIM model entities without contextual information or dependencies on their surroundings. To address this limitation, Fig. 3 presents a workflow that enables the identification of surrounding elements and converts their IFC geometry into mesh and then performs points sampling, resulting in a point cloud.

The process begins with opening and parsing the model using the IfcOpenShell library, which serves as an efficient bridge for integrating IFC data with other analytical tools [48]. Once the model is successfully opened, each element is preprocessed to extract bounding box information, used as a quick and efficient way to determine components' proximity. For a given element with a known bounding box (Step 1), the bounding box is expanded by one meter in each direction to identify surrounding components, as shown in Step 2 in Fig. 3. This extension facilitates the detection of nearby elements that may not directly touch the target entity but exist within its immediate vicinity. The condition



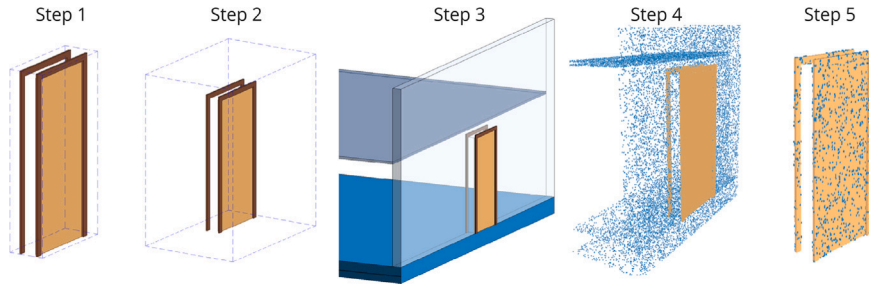


Fig. 3. Illustration of the process of converting element and context geometry to point cloud representation.

for the intersection of two bounding boxes,  $A$  and  $B$ , is determined using Eq. (1):

$$\begin{cases} x_{\min}^A \leq x_{\max}^B \text{ and } x_{\max}^A \geq x_{\min}^B \\ y_{\min}^A \leq y_{\max}^B \text{ and } y_{\max}^A \geq y_{\min}^B \\ z_{\min}^A \leq z_{\max}^B \text{ and } z_{\max}^A \geq z_{\min}^B \end{cases} \quad (1)$$

This intersection check is essential to determine which elements are contained within the expanded bounding box, ensuring that only all relevant context elements are included for analysis without analysing and wasting computational power for components irrelevant to an analysed element. After identifying the surrounding components, as shown in Step 3 in Fig. 3, the IFC element is converted to mesh geometry before being transformed into a point cloud using the Trimesh library [49]. This conversion improves performance by simplifying complex geometric representations into a format that allows for efficient point sampling. Point sampling is performed on all neighbouring elements. Still, it is limited to the boundaries of the extended bounding box around the given element, as depicted in Step 4 in Fig. 3, resulting in a context point cloud with 16 384 XYZ points.

The proposed workflow further includes converting the target element itself into a point cloud, ensuring uniform data representation across both element and context components. After the bounding box expansion and surrounding element detection, the geometry of the target element is sampled independently to yield a point cloud representation with 2048 XYZ points, as shown in Step 5 in Fig. 3.

To enable separate normalisation and processing of the element and context point clouds, each point in the context point cloud is assigned an additional feature reflecting its connection to the target element. Specifically, each context point receives a distance-based weight that indicates its relative influence on the element. Let  $el\_pc$  denote the point cloud of the target element, while  $ctx\_pc$  refers to the surrounding context points.

For each point  $p_i$  in  $ctx\_pc$ , the Euclidean distance to the closest point  $q$  in  $el\_pc$  ( $el\_pc$ ) is using Eq. (2)

$$d_i = \min_{q \in el\_pc} \|p_i - q\| \quad (2)$$

where  $d_i$  represents the minimum distance between the  $i$ th point in  $ctx\_pc$  and any point  $q$  in  $el\_pc$ .

These distances are then normalised to a range between 0 and 1, where shorter distances correspond to a higher impact. The normalised impact  $I_i$  for each point  $p_i$  is given by Eq. (3):

$$I_i = 1 - \frac{d_i}{\max(d)} \quad (3)$$

where  $\max(d)$  is the maximum distance across all context points. In this way, points closer to the element have values closer to 1, while points farther away approach 0. This additional dimension  $I_i$  is incorporated into the context point cloud, allowing the model to account for the relative importance of each point based on its proximity to the target element. Upon successful conversion, both the element and its context are stored in the database for use in subsequent dataset creation stages.

#### 4.1.2. Normalisation

Data normalisation is applied to an element and its context point clouds to ensure efficient training and maintain a consistent feature range across all input parameters. This process stabilises the neural network's operating range, enhancing the comparability across different component classes and improving the overall robustness of the model. Each point in the point cloud is represented as a vector with three elements corresponding to the  $x$ ,  $y$ , and  $z$  coordinates. To normalise these points to a range of  $[0, 1]$ , the minimum and maximum values for each coordinate across all points are calculated using Eq. (4):

$$\text{minimum\_values}_i = \min(p_i), \quad \text{maximum\_values}_i = \max(p_i) \quad (4)$$

where  $p_i$  represents the values of the point cloud along each axis  $i$  ( $i \in \{x, y, z\}$ ). From these, the range along each axis is defined as shown in Eq. (5):

$$\text{range\_along\_axes}_i = \text{maximum\_values}_i - \text{minimum\_values}_i \quad (5)$$

The normalisation factor, which scales the points into the desired range, is the maximum of these ranges following Eq. (6):

$$\text{normalisation\_factor} = \max(\text{range\_along\_axes}_i) \quad (6)$$

To bring points into the  $[0, 1]$  range, the minimum values are subtracted from each coordinate, and the result is divided by the normalisation factor using Eq. (7):

$$p_i^{\text{norm}} = \frac{p_i - \min_i}{\text{normalisation\_factor}} \quad (7)$$

After scaling, a shift is applied to ensure the points are centred within the  $[0, 1]$  range on each axis. This shift is necessary because the ranges along each axis may differ, and by shifting, the points become symmetrically distributed within the normalised range. The change for each axis is calculated using Eq. (8):

$$\text{center\_shift}_i = \frac{1 - \frac{\text{range\_along\_axes}_i}{\text{normalisation\_factor}}}{2} \quad (8)$$

Finally, the normalised and centred coordinates can be calculated for each point using Eq. (9):

$$p_i^{\text{norm}} + \text{center\_shift}_i \quad (9)$$

The effect of applying this normalisation technique to an *ifcStair-Flight* element is illustrated in Fig. 4. The figure shows that the shape and relationships between points are preserved, with the only changes being the position and scale of the axes.

#### 4.1.3. Augmentation

The purpose of the augmentation process is to enhance the variability of the dataset, enabling the model to learn more generalised patterns and preventing overfitting to specific orientations or spatial configurations. The proposed framework uses two augmentation techniques.

The first type of augmentation is jittering. It introduces slight, random variations in the spatial position of each point within the point

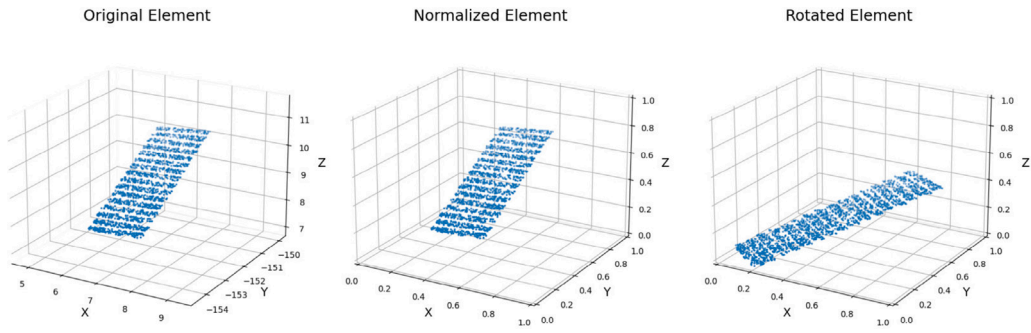


Fig. 4. Comparison of the 3D point cloud of the original, normalised, and rotated elements. The original element (left) shows the raw data in its initial coordinates, with unscaled dimensions and uncentered spatial orientation. The normalised element (centre) is scaled to a unit range [0, 1] and translated to the centre. The rotated element (right) demonstrates the application of a 45-degree rotation on the normalised data.

cloud, effectively preventing the model from becoming overly sensitive to exact point positions and promoting generalisation by encouraging the model to recognise patterns across slightly different spatial arrangements. This process adds a small, controlled amount of noise, ensuring that the overall shape and structure of the object remain intact. During data preparation, up to 0.5% jittering was applied relative to the size of the entire element.

The second type of augmentation applied while preparing the dataset is rotation. This transformation introduces orientation variability, allowing the model to recognise features from different perspectives and improve its generalisation capability. By applying rotation, the model becomes less dependent on specific orientations, preventing it from associating certain features with a fixed directional axis.

The rotation is applied around the Z-axis, with a random angle chosen between  $-120^\circ$  and  $120^\circ$ . Rotation around the Z-axis preserves the natural orientation of each component, avoiding unrealistic transformations, such as flipping elements upside down, that could distort their intended function. To retain spatial consistency, the same rotation angle is applied to both the element and its surrounding context, ensuring the correlation between them remains intact. Fig. 4 demonstrates this augmentation on an *ifcStairFlight* entity: the normalised component in its original orientation (centre) is rotated by an angle of  $45^\circ$  (right), illustrating the resulting variability while preserving logical alignment and dependency between points.

#### 4.1.4. Dataset preparation

In preparing the dataset, a list of 21 the most frequently occurring IFC classes in the available models was selected for training: *ifcBeam*, *ifcColumn*, *ifcCurtainWall*, *ifcDoor*, *ifcEnergyConversionDevice*, *ifcFlowController*, *ifcFlowFitting*, *ifcFlowMovingDevice*, *ifcFlowSegment*, *ifcFlowTerminal*, *ifcFlowTreatmentDevice*, *ifcFooting*, *ifcFurnishingElement*, *ifcPlate*, *ifcRailing*, *ifcRoof*, *ifcSlab*, *ifcStair*, *ifcStairFlight*, *ifcWall*, *ifcWindow*.

From each class, 170 normalised elements were chosen based on a set of extracted elements from IFC models as described in Section 4.1.1.1. Ensuring an equal number of elements per class helps maintain unbiased model training across all categories. These components were then divided into training, validation, and testing datasets with an 8:1:1 split ratio, resulting in 2856 components for training and 357 for both validation and testing.

The dataset was divided into these subsets for the following purposes:

- **Training set:** Used to train the neural network and optimise gradients.
- **Validation set:** Used during training to monitor stability and prevent overfitting, allowing the model to evaluate data it has not seen before and gauge generalisation capability.
- **Testing set:** Reserved for final evaluation, assessing performance on entirely unseen data to measure the model's actual learning accuracy.

For consistency and comparability, the same components were used across datasets for both PointNet and ContextNet models, enabling a fair comparison of their performance and learning behaviour. To enhance robustness, the training set was augmented five times, increasing the total number of training elements to 17,136. Each element in the validation and testing sets was augmented once, resulting in 714 components for each of these sets.

## 4.2. Neural network architecture

The proposed architecture for ContextNet integrates multiple encoders, including a PointNet-based encoder, to perform contextual classification of building components using point cloud data. PointNet is used as the first encoder to extract local features from the point cloud by handling unordered point sets while ensuring invariance to input permutations. The point cloud data is created for both the building components and their context, allowing the model to understand the relationship between the elements and their surroundings. This section describes the architectural components, their configuration, and how point cloud data is combined with contextual information for enhanced classification accuracy. Fig. 5 illustrates the overall architecture.

### 4.2.1. Architecture overview

The model consists of two feature extraction encoders and dense layers for classification. The first encoder, denoted as Encoder 1 in Fig. 5, extracts local features from element point cloud data. The second one, denoted as Encoder 2 in Fig. 5, extracts contextual features from the point cloud representing surrounding elements. These features are combined and forwarded through the next set of layers, resulting in the classification output.

### 4.2.2. Feature extraction from point cloud data

Encoder 1, presented in Fig. 5, operates on the building element point cloud data, which includes 2048 points with three features representing (x, y, z) point coordinates. This encoder uses an Input Features Transformer (marked with a solid line) block, as detailed in Fig. 5, to compute transformation matrices that align the input through matrix multiplication. The TNet module aims to learn an affine transformation that aligns the input data to a canonical space, making the features invariant to spatial transformations. Next, the model increases the number of features from three to 64 using a convolutional layer and applies another TNet transformation, denoted as Element Features Transform in Fig. 5. This process ensures that the features remain invariant to spatial transformations, such as rotation and translation. The ML Model architecture is based on PointNet, utilising similar layers and transformations to capture geometric features from point cloud data.

Encoder 2, presented in Fig. 5, operates on the context point cloud data of the building element. The context point cloud representation

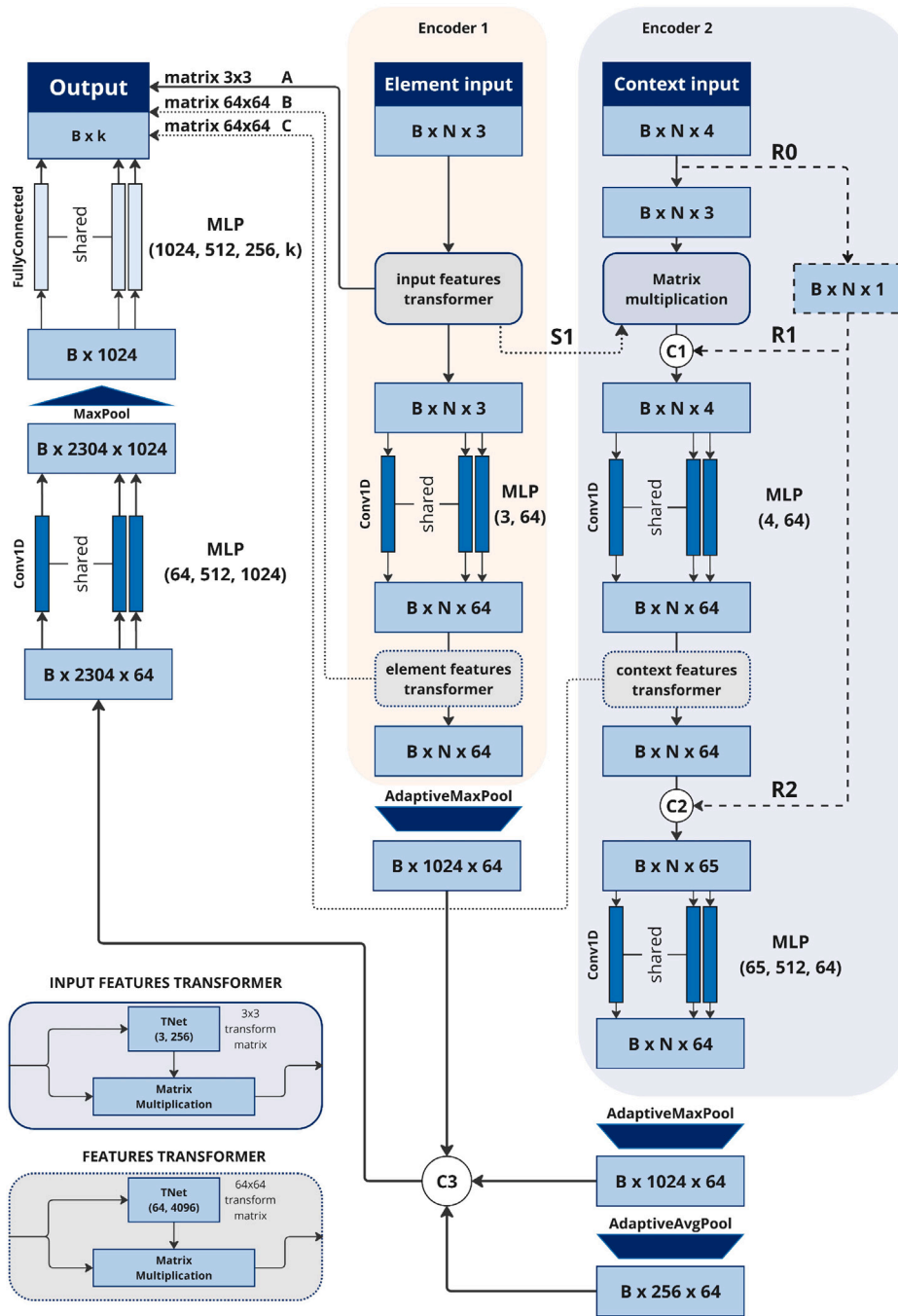


Fig. 5. ContextNet architecture with dual encoders: Encoder 1, based on PointNet, processes the primary element features, while Encoder 2 extracts contextual information from surrounding data points. The features from both encoders are concatenated along the point indices and then processed through additional layers, resulting in the final output layer for classification. TNet modules are employed for the spatial alignment of input features.

contains 16384 points and includes an additional dimension that describes the normalised distance between the object and context points ( $x, y, z, d$ ). This distance information represents the impact of a contextual point on the element. The distance feature is detached from the context representation vector, as denoted by R0 in Fig. 5. The resulting three features representing the context points coordinates ( $x, y, z$ ) is then multiplied by the element's input feature transformation matrix, which is shared between the encoders, as indicated by S1 in Fig. 5. The purpose of using the element transformation matrix is to keep the context aligned with the element. The distance data is excluded from matrix multiplication, as it typically does not require

transformation since it represents a contextual relationship rather than a spatial coordinate; preserving it separately ensures it retains its intended meaning.

In the subsequent step, the distance features are concatenated (C) with the context coordinates, as denoted by the R1 dashed line in Fig. 5. The encoder then uses a convolutional layer similar to the first encoder but with a higher dimensionality to account for the distance feature. A TNet block is subsequently applied to ensure that the context features remain invariant to spatial transformations. Finally, the results are concatenated with the second distance feature concatenation (C), as denoted by R2 in Fig. 5.

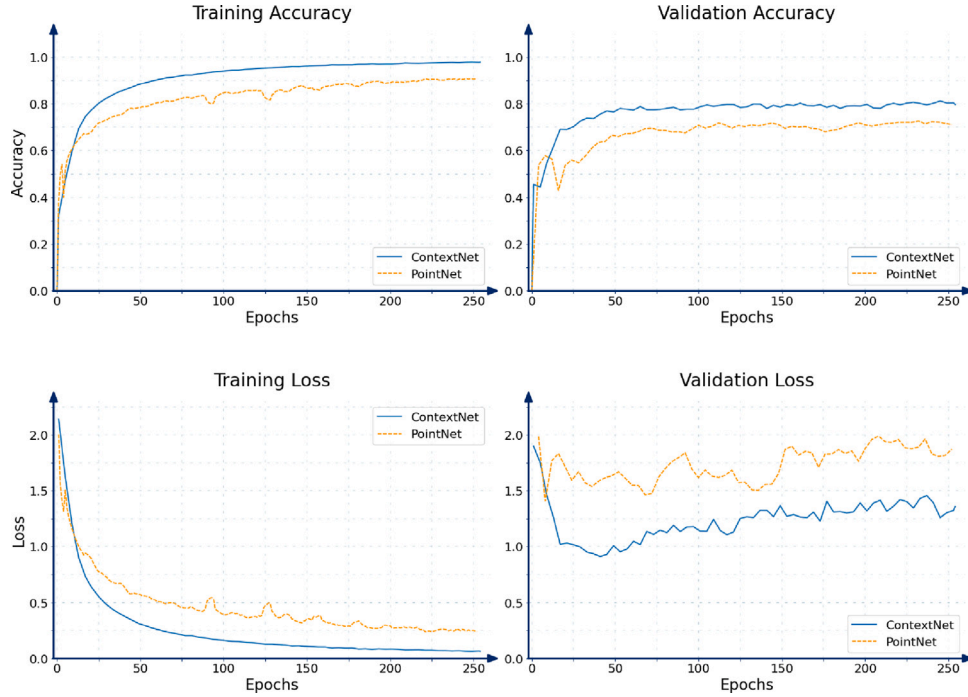


Fig. 6. Training and Validation Performance of PointNet over 256 Epochs. The figure displays the training accuracy (top left), validation accuracy (top right), training loss (bottom left), and validation loss (bottom right).

#### 4.2.3. Integration of features and pooling operations

Features from both encoders are integrated to form a unified representation. For Encoder 1, an adaptive max pooling operation is applied to extract the most dominant local features from the element point cloud, resulting in a compact and informative representation. For Encoder 2, after convolutional layers, both adaptive max pooling and adaptive average pooling are used. The max pooling operation captures the most prominent context features, while average pooling provides a broader statistical characterisation of the surrounding environment. This combined approach balances the retention of significant geometric features with a more comprehensive representation of contextual information.

Unlike the original PointNet approach, which employs a single global max pooling operation to enforce order invariance, the design choice to use adaptive pooling was motivated by extensive experimentation. Over 500 different network configurations were evaluated, testing various strategies for fusing element and context features. The final architecture maintains a structured ratio of 1:1 between the element and context representations by extracting 1024 features from the element and 1024 features from the context via max pooling. Additionally, 256 extra features are extracted through adaptive average pooling to capture a more generalised statistical representation of the dominant context features.

The motivation behind this choice was to retain a richer feature distribution from the context while preserving the robustness of the network to input point permutations. While global pooling simplifies the feature space to a single aggregated vector, it leads to the loss of intermediate details that are crucial for distinguishing between geometrically similar elements in different contexts. By maintaining a structured feature extraction mechanism, the model ensures that spatial relationships are incorporated into the classification process.

The architecture remains robust to point permutations due to the shared Multi-Layer Perceptron (MLP) layers and the use of symmetric pooling operations. These components, aligned with the core principles of PointNet, guarantee that the model remains invariant to input order. Additionally, data shuffling during training further reinforces this robustness.

Finally, the concatenation step integrates 1024 max-pooled features and 256 average-pooled features from the context with 1024 max-pooled features from the element. This structured integration allows both local element features and global context information to contribute effectively to classification. The fused features then pass through a series of convolutional layers, followed by a final max pooling operation and fully connected layers. This produces a classification output of size  $k$ , where  $k$  represents the number of classes.

#### 4.2.4. Loss

The loss function for ContextNet is calculated as the sum of the classification loss (CrossEntropyLoss) and an orthogonality regularisation term. The classification loss measures the discrepancy between the predicted and actual labels, while the orthogonality regularisation encourages the transformation matrices to be as close to orthogonal as possible. For each transformation matrix, denoted as  $T$ , the orthogonality loss is defined by Eq. (10):

$$\text{ort\_loss} = \|T \cdot T^T - I\|_F^2 \quad (10)$$

where  $\|\cdot\|_F$  represents the Frobenius norm, measuring the deviation of  $T \cdot T^T$  from the identity matrix  $I$ . This regularisation is inspired by the TNet module in PointNet, which stabilises the network by maintaining feature alignment.

The total loss for ContextNet, denoted as  $t\_loss$ , combines the classification loss with orthogonality regularisation terms applied to three transformation matrices: the input  $3 \times 3$  feature matrix (A), element  $64 \times 64$  features matrix (B), and context  $64 \times 64$  features matrix (C), as illustrated in Fig. 5. This is expressed as shown by Eq. (11):

$$t\_loss = \text{loss} + \alpha_{el} \cdot (\text{ort\_loss}_{M3 \times 3} + \text{ort\_loss}_{M64 \times 64}) + \alpha_{ctx} \cdot \text{ort\_loss}_{M64 \times 64} \quad (11)$$

where  $\text{loss}$  represents the classification loss, and  $\alpha_{el}$  and  $\alpha_{ctx}$  are coefficients set to 0.0001 and 0.0002, respectively, to balance the contributions of element and context orthogonality losses.



#### 4.2.5. Model training

The ContextNet model is trained using a batch size of 64, with shuffling applied to the training dataset to improve data diversity and reduce overfitting risks. The model parameters are optimised using the Adam optimiser with a fixed learning rate of 0.001. A small weight decay of 0.00001 is used, which controls the growth of model weights and helps prevent the network from becoming over-fitted to the training data.

Validation is performed every four epochs to monitor the model's progress and save the best-performing checkpoint based on validation accuracy. The training process consists of 256 epochs, after which the final evaluation of the test dataset uses the checkpoint with the highest validation accuracy.

In addition to ContextNet, the PointNet model is trained using the same parameters: a learning rate of 0.001, weight decay of 0.00001, batch size of 64, and dataset shuffling. This consistent setup enables a direct comparison of the performance of the two models based on their different architectures.

## 5. Results

### 5.1. PointNet training results

The model was trained in an Ubuntu 22.04 environment equipped with an NVIDIA A100-SXM4-40 GB GPU. Using the hyperparameters specified in Section 4.2.5, the training process required a total of 2433.8 s, achieving a final accuracy of 73.18% on the test dataset. The PointNet model contains 21 181 082 parameters, which forward pass for a single element takes 0.156 ms, resulting in 6410 processed elements per second.

The training and validation accuracy results for PointNet over 256 epochs are shown in Fig. 6. Over the initial ten epochs, PointNet demonstrates a rapid increase in both training and validation accuracy, reaching a peak validation accuracy of 75.16%. Following this early phase, accuracy stabilises with minor fluctuations, suggesting that the model quickly reaches its learning capacity and maintains consistent performance.

The training and validation loss curves, also depicted in Fig. 6, show that PointNet efficiently reduces loss during the early epochs, which then levels out as the model converges. These results illustrate the model's ability to learn effectively from the training data in the initial stages and maintain steady performance as it approaches its optimal solution.

The highest accuracy achieved by PointNet on the test dataset is observed for the *IfcStairFlight*, *IfcFurnishingElement*, and *IfcEnergyConversionDevice* classes, each reaching 100% accuracy. The model's lowest accuracy was recorded for the *IfcWall* class, at 29%, with lower performance also noted for the *IfcCurtainWall*, *IfcFlowTerminal*, and *IfcSlab* classes, each achieving 56%. The classification performance across classes is summarised in the bottom section of the confusion matrix in Fig. 7. Rows in the confusion matrix represent the true class labels, while columns indicate predicted class labels, with each cell value showing the ratio of predictions per class. This view provides an overview of PointNet's accuracy and misclassification tendencies across categories.

### 5.2. ContextNet model results

The neural network was trained using the same environment (Ubuntu 22.04 equipped with an NVIDIA A100-SXM4-40 GB GPU) with hyperparameters specified in Section 4.2.5. The training process required 15697 s, achieving a final accuracy of 82.19% on the test dataset. ContextNet model contains 41 240 026 parameters, for which the forward pass for a single element takes 0.719 ms, resulting in 1390 preprocessed elements per second.

The training and validation accuracy results for ContextNet over 256 epochs are shown in Fig. 6. The blue lines indicate ContextNet's steady improvement in accuracy over time, stabilising around 80% and peaking at 83.10%. This consistent learning pattern suggests that ContextNet effectively leverages contextual information, resulting in reliable performance over the course of training.

The training and validation loss for ContextNet, also shown in Fig. 6, continues to decrease steadily throughout the training process, highlighting the model's effective use of contextual data to minimise error.

On the test dataset, ContextNet achieved the highest accuracy for the *IfcStair* and *IfcEnergyConversionDevice* classes, each with 100% accuracy. Additionally, high accuracy was observed for the *IfcStairFlight*, *IfcRailing*, and *IfcFurnishingElement* classes, achieving 97% accuracy. Conversely, the lowest performance was recorded for the *IfcFlowFitting* class, with an accuracy of 47%, while the *IfcWall* and *IfcFlowSegment* classes showed accuracies of 59% and 65%, respectively. The top section of the confusion matrix in Fig. 7 provides an overview of ContextNet's classification performance, with each cell value representing the proportion of predictions per class and darker diagonal cells indicating higher accuracy.

## 6. Discussion

### 6.1. Integration of surrounding information in object classification tasks

This study proposes a framework that integrates the geometric characteristics of individual elements with their surrounding context, represented as point clouds. This dual-input system allows for the coherent and efficient incorporation of contextual information in object classification tasks.

The framework enables efficient, customisable integration of contextual information. The concept is adaptable to varying sizes and numbers of elements, making it applicable across a wide range of use cases. It is not constrained by the scale of the primary component or its surroundings, ensuring flexibility in handling diverse scenarios. By leveraging point clouds, the framework maintains a constant data shape, simplifying processing and ensuring consistency throughout the classification pipeline. Additionally, the modular design allows for seamless adjustments to the density of point cloud data, optimising computational resources without compromising accuracy. One of the main benefits of this method is its scalability. The number of points representing both the main element and its context can be fine-tuned based on the scope of interest and specific domain requirements. For instance, in infrastructure projects where components are larger, the framework can incorporate more context points to capture broader spatial relationships. This adaptability ensures its applicability across diverse project scales, from small interior spaces to extensive infrastructure environments. It supports more accurate and contextually aware classification in various scenarios within the AEC industry.

### 6.2. Classification accuracy

The ContextNet architecture, as illustrated in Fig. 5, demonstrates a significant improvement in classification accuracy over the PointNet model (Fig. 2), achieving a 9.01% increase in average accuracy. To ensure a fair and reliable comparison, both models were trained on identical datasets with the same component count and class distribution. This consistency in dataset design underscores the influence of the model architecture itself on classification performance rather than variations in training data.

The results indicate that ContextNet achieves higher accuracy across a majority of the 21 IFC classes used in the study. This improvement highlights the advantage of incorporating contextual information in distinguishing between building components with similar geometric features but differing functions, which is especially valuable in complex

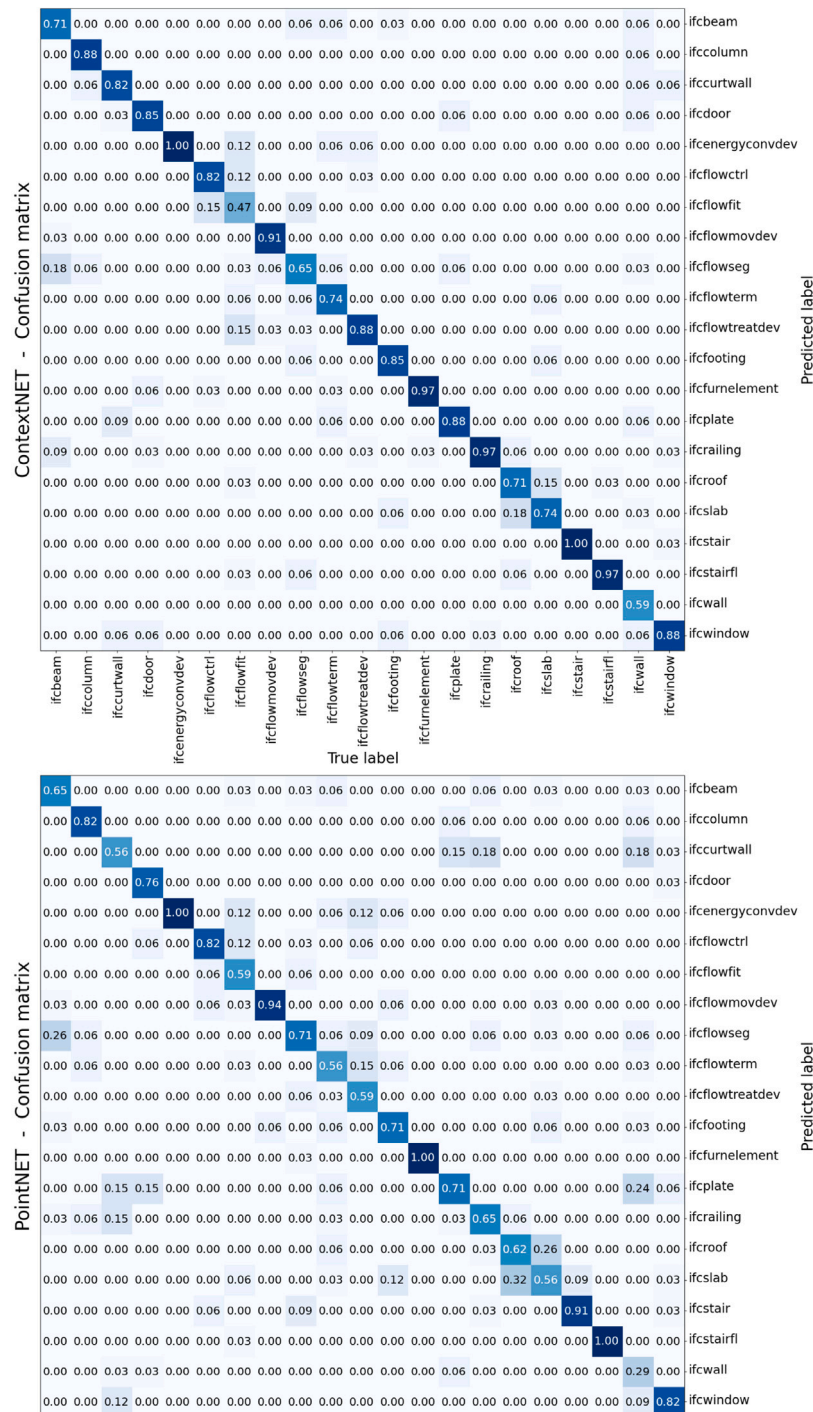
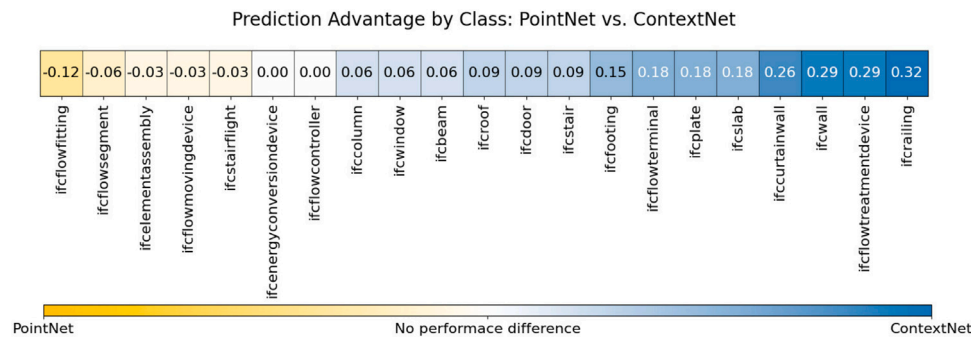


Fig. 7. Confusion matrices for ContextNet (top) and PointNet (bottom) models, showing classification performance across multiple classes. Diagonal cells represent correct predictions, with darker shades indicating higher accuracy, while off-diagonal cells highlight misclassifications.

or ambiguous classifications. For instance, classes such as *ifcBeam* and *ifcFlowSegment* might share similar shapes but serve different roles in a structure, making them challenging to differentiate based solely on isolated geometry. The surrounding context provided by ContextNet allows the model to consider spatial relationships, such as proximity to structural elements or other pipes, which aids in more accurate classification.

To directly compare model performance, Fig. 8 provides a head-to-head comparison across classes, sorted from the most PointNet-favoured (left) to the most ContextNet-favoured (right).

PointNet shows the greatest advantage in classes like *ifcFlowFitting* and *ifcFlowSegment*, outperforming ContextNet by 12 and 6 percentage points, respectively. However, as illustrated in Fig. 7, both models struggle with the classification performance of *ifcFlowFitting* elements, suggesting that part of the dataset may benefit from more diverse geometrical representations and additional investigation of components characteristic or splitting them into pipe related and duct related components, ensuring that more geometrical coherence between them is provided.



**Fig. 8.** Performance difference between PointNet and ContextNet across various IFC classes. Positive values (in blue) indicate an accuracy advantage for ContextNet, while negative values (in orange) favour PointNet. Each class is represented by a labelled square, showing the percentage point difference in accuracy between the models.

This analysis underscores the importance of context in enhancing classification accuracy for building components. Despite some underperforming categories, the overall accuracy improvement from 73.18% with PointNet to 82.19% with ContextNet highlights the significant impact of contextual information. The use of identical datasets for both models ensures a fair comparison, confirming that the observed improvements are due to architectural differences rather than variations in training data.

In contrast to the approach of Austern et al. [37], which leverages explicit information from neighbouring elements, ContextNet is designed to work solely based on the geometry of an element and closes adjacent elements. While Austern et al. [37] method is limited to ten Autodesk Revit categories, ContextNet tackles a significantly more complex task by classifying 21 distinct classes relying only on geometric without any user-based information.

A closely related in terms of inclusion of surrounding elements is presented by Luttun and Krijnen [29], who developed IfcVoxelNet. Although IfcVoxelNet achieves impressive classification results—over 95% for walls and windows and 98% for railings, it underperforms for beams, columns and stairs with accuracy from 0 to 0.5%. Moreover, the IfcVoxelNet is trained on a simplified dataset with only nine classes derived from public IFC files. In comparison, ContextNet's performance for 21 classes demonstrates its robustness in handling a broader and more challenging set of building elements.

### 6.3. Model efficiency

When choosing a machine learning model, an important consideration is not only its accuracy but also the trade-off between model complexity and computational efficiency. While accuracy remains a primary objective, balancing it with practical aspects such as processing speed and data requirements can significantly impact the model's application feasibility for real-world applications.

The compact PointNet architecture contains approximately 21 million parameters, whereas ContextNet's design increases this to around 41 million parameters, representing a 94.7% increase in network complexity. This added complexity allows ContextNet to capture richer contextual information, but it comes at a computational cost.

In addition to the architectural differences, the input data requirements for training these models differ substantially. PointNet's input consists of element representations with 2048 points, each having 3 features. In contrast, ContextNet requires this same element representation along with additional context data of 16 384 points with four features, which demands considerably more data processing during training. As a result, training ContextNet takes approximately 6.45 times longer than PointNet.

In practical terms, this increased complexity also affects prediction speed. While the fully trained PointNet model processes each element in 0.156 ms, ContextNet requires 0.719 ms per element, making its prediction time 4.6 times longer. Thus, while ContextNet achieves higher

accuracy, these results underscore the trade-off between enhanced performance and the increased computational resources required to achieve it.

### 6.4. Data-related challenges

A critical factor in training and validating machine learning models is the availability and quality of datasets. Current publicly available datasets predominantly contain information on stand-alone components, isolated from their surrounding context and often without reference to the original model. To prepare datasets for this study, it was essential to source a wide variety of models that represented a diverse range of component classes. This approach, after thorough data curation, facilitated effective model training and increased generalisability.

However, challenges arise due to varying modelling practices across different model owners, which complicate dataset preparation, especially for context-dependent analysis. Some models provide federated representations that encompass all domains — architecture, structure, and MEP design — while others separate these domains into individual files or combine them inconsistently. This variability introduces added complexity in data curation, as inconsistencies between domains can lead to ambiguities in the integrated point cloud representation and confusion while training.

Additionally, while numerous open-source models are available, there remains a reluctance among professionals in the AEC industry to share models publicly due to intellectual property concerns, limiting the scope of accessible data and reducing the available data scope for AI model training and testing.

Apart from dataset availability, the incorporation of the surrounding context for each component notably increases processing time. While analysing single elements in isolation is relatively computationally efficient, integrating contextual information requires a more complex and computationally expensive approach. Capturing and converting the surrounding elements into a point cloud involves identifying, extracting, and processing multiple adjacent components rather than focusing solely on a single element. This additional processing increases computational demands, as each neighbouring element must be translated into a compatible data structure and combined with the target element's data. Therefore, the extraction of data from IFC models is also an important consideration which should be taken into account together with AI model complexity and their forward pass time.

### 6.5. Usage of the AI model in AEC workflow

While the ContextNet model is trained, it can be used to support BIM model quality checks and make the validation process more efficient. Depending upon the processes declared in the BIM Execution Plan (BEP) and project structure, the validation and quality check procedures might be applied for various purposes. The first possible usage is a tool for anomaly detection, similar to our previous work et al. [16]. The



PointNet model could be replaced with a model with better accuracy to validate data within a model and highlight classes which are the most prompt to be wrongly classified.

An alternative application could follow assumptions presented by Emunds et al. [26] by integrating the AI model directly into the BIM data exchange and enrichment workflow. In this scenario, the model is employed during the export or in real-time as BIM objects are generated, automatically classifying and verifying the semantic labels of each element. This approach enables immediate detection and correction of misclassifications, ensuring that the BIM model contains correct elements. One of the potential beneficiary groups of users are BIM Coordinators who ensure that the data included in models comply with the standards and requirements of the BEP during certain data drops. Wrapping the neural network with such high accuracy and a significant number of classes could significantly reduce the time required for manual checks and highlight the elements most prone to error.

## 6.6. Current limitations

Despite the framework's benefits and its potential for extensive testing and robustness, several limitations should be considered. One primary limitation is that this study focuses on building components related to Architectural, Structural, and MEP models without extending the AI model to infrastructure components. Neglecting of infrastructure components is caused by their availability and lack of multiple open-source models.

Additionally, the framework uses a fixed one-meter extension to the element bounding box in each direction without exploring alternative sampling sizes. This choice may require a higher number of context points than necessary, potentially impacting both model performance and data processing efficiency. A convergence study could be conducted to systematically investigate how variations in the bounding box size and the number of context points influence the model's accuracy and computational cost. Such an approach would help identify the optimal configuration, ensuring a balance between performance and resource efficiency.

The last limitation is the number of components used for training. To ensure a balanced dataset, the minimum number of instances for any given class dictates the sample size, potentially limiting the diversity of data available for training. Ensuring an equal number of entities per class during training is essential to prevent model bias towards more common classes. While weighted loss functions, such as CrossEntropy or NLL Loss, can mitigate class imbalance when the number of elements is unequal, the incorporation of orthogonal regularisation complicates this process. Consequently, to improve class diversity and model robustness, it would be ideal to source at least a thousand entities per class from a wide range of IFC models, leveraging diverse data modelling approaches. This would not only address class imbalance but also ensure that the model is exposed to a broader spectrum of geometric and contextual variations, ultimately enhancing its ability to generalise to real-world scenarios.

## 7. Conclusions

### 7.1. Summary

This study presents ContextNet, a novel deep-learning neural network architecture designed to improve the classification accuracy of building components by incorporating contextual information in the form of point cloud data. By leveraging the surrounding geometry of each component, ContextNet demonstrates a notable increase in classification accuracy compared to the stand-alone component analysis of PointNet. Specifically, the integration of contextual data allows ContextNet to distinguish more effectively between classes with similar geometric features, enhancing its performance across a diverse range of building components.

The results highlight that while the addition of contextual information significantly enhances classification accuracy, it also introduces computational costs. ContextNet's model complexity and data requirements are greater than those of PointNet, impacting both training and inference times. This trade-off underscores the importance of balancing model performance with computational efficiency, particularly for real-time or resource-constrained applications in the AEC industry.

Additionally, this study identifies key challenges associated with contextual data integration, including dataset diversity, data processing time, and inconsistencies in modelling practices. These limitations suggest avenues for future work, such as optimising context sampling techniques, exploring adaptive context regions, test other classification systems and increasing the diversity of datasets to capture a broader range of component types and configurations.

### 7.2. Future work

Future work will address three key limitations identified in this study, focusing on enhancing the framework's adaptability and robustness. First, the model will be trained on infrastructure components using the IFC 4.3 schema, validating its applicability beyond building components and testing its performance in infrastructure domains. Second, upcoming studies will expand the training, testing, and validation datasets to include not only open-source models but also data from commercial projects, ensuring a more comprehensive and diverse dataset. Finally, a convergence study will be conducted to identify the optimal bounding box size and the number of context points required for analysis. This study has the potential to significantly improve model accuracy and processing efficiency, making the framework more practical and scalable for implementation in commercial projects.

Additionally, future research will focus on testing the model's performance in classification within other systems. The next study will evaluate the model using components classified under Uniclass and Unifomat, assessing its ability to handle classification systems with significantly more categories than those in the current dataset. This will involve creating datasets labelled according to these standards and analysing how well the model generalises beyond IFC-based categorisation. Training the model across multiple classification systems will allow for a comparative analysis of its performance across different labelling conventions and assess its adaptability to various industry standards.

### CRedit authorship contribution statement

**Wojciech Teclaw:** Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Data curation, Conceptualization. **Artur Tomczak:** Writing – review & editing, Writing – original draft, Software, Resources, Investigation, Data curation, Conceptualization. **Mateusz Kasznia:** Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Conceptualization. **Marcin Luczkowski:** Writing – review & editing, Writing – original draft, Investigation, Conceptualization. **Nathalie Labonnote:** Writing – review & editing, Supervision, Project administration, Funding acquisition. **Eilif Hjelseth:** Supervision, Project administration.

### Declaration of Generative AI and AI-assisted technologies in the writing process

In developing this manuscript, ChatGPT, a generative AI language model, was utilised to improve the readability and grammar syntax.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.



## Data availability

Data will be made available on request.

## References

- [1] Mohd Rafeeq Ur Rahman, Mohammed Iqbal Khatib, P. Seema Rani, Shahin Shaikh, Gunashekar. G, Effect of computer aided drafting on manual drafting skills, *Int. J. Innov. Technol. Explor. Eng.* 8 (11) (2019) 3012–3014, <http://dx.doi.org/10.35940/ijitee.K2315.0981119>.
- [2] André Borrmann, Markus König, Christian Koch, Jakob Beetz, Building information modeling: Why? What? How? in: André Borrmann, Markus König, Christian Koch, Jakob Beetz (Eds.), *Building Information Modeling*, Springer International Publishing, Cham, ISBN: 978-3-319-92861-6, 2018, pp. 1–24, [http://dx.doi.org/10.1007/978-3-319-92862-3\\_1](http://dx.doi.org/10.1007/978-3-319-92862-3_1).
- [3] Charles Eastman, An Outline of the Building Description System, Technical Report Research Report No. 50, Carnegie-Mellon University, Institute of Physical Planning, Pittsburgh, PA, 1974, URL <https://eric.ed.gov/?id=ED113833>. (Accessed 16 November 2023).
- [4] Charles Eastman, The use of computers instead of drawings in building design, *AIA J.* (1975) URL <https://eric.ed.gov/?id=EJ114213>. (Accessed 16 November 2023).
- [5] International Organization for Standardization, Organization and digitization of information about buildings and civil engineering works, including building information modelling (BIM). information management using building information modelling, 2018, <http://dx.doi.org/10.3403/BSENISO19650>.
- [6] Deloitte, State of Digital Adoption in the Construction Industry 2024, Technical Report, Deloitte, 2024, URL <https://www.deloitte.com/au/en/services/economics/analysis-state-digital-adoption-construction-industry.html>. (Accessed 17 August 2024).
- [7] buildingSMART International, Global IFC Mandates, Technical Report, BuildingSMART, 2024, URL <https://www.buildingsmart.org/wp-content/uploads/2024/10/IFC-Mandate-Final-1.pdf>. (Accessed 01 June 2024).
- [8] Tran Duong Nguyen, Sanjeev Adhikari, The role of BIM in integrating digital twin in building construction: A literature review, *Sustainability* 15 (13) (2023) 10462, <http://dx.doi.org/10.3390/su151310462>.
- [9] Yue Pan, Limao Zhang, A BIM-data mining integrated digital twin framework for advanced project management, *Autom. Constr.* (ISSN: 09265805) 124 (2021) <http://dx.doi.org/10.1016/j.autcon.2021.103564>.
- [10] Michael Belsky, Rafael Sacks, Ioannis Brilakis, Semantic enrichment for building information modeling, *Comput.- Aided Civ. Infrastruct. Eng.* (ISSN: 1093-9687) 31 (4) (2016) 261–274, <http://dx.doi.org/10.1111/mice.12128>.
- [11] Hossain Md Aslam, Haron Ahmad Tarmizi, Intelligent BIM record model for effective asset management of constructed facility, *MATEC Web Conf.* 195 (2018) 06004, <http://dx.doi.org/10.1051/mateconf/201819506004>.
- [12] Chi Zhang, Jakob Beetz, Matthias Weise, Interoperable validation for IFC building models using open standards, *J. Inf. Technol. Constr.* 20 (2015).
- [13] Jin Wu, Jiansong Zhang, Luciana Debs, Model validation for automated building code compliance checking, in: *Proceedings of the ASCE Construction Research Congress, American Society of Civil Engineers*, Reston, VA, 2022, pp. 640–650, <http://dx.doi.org/10.1061/9780784483961.067>, Accessed: 2024-03-12.
- [14] Jin Wu, Jiansong Zhang, New automated BIM object classification method to support BIM interoperability, *J. Comput. Civ. Eng.* 33 (5) (2019) 04019033, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0000858](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000858).
- [15] Yohann Schatz, Bernd Dömer, Semi-automated creation of IFC bridge models from point clouds for maintenance applications, *Front. Built Environ.* (ISSN: 2297-3362) 10 (2024) <http://dx.doi.org/10.3389/fbuil.2024.1375873>.
- [16] Wojciech Teclaw, Artur Tomczak, Marcin Luczkowski, Neural network for IFC class recognition, in: *The 41st CIB W78 Conference on Information Technology for Construction*, 2024.
- [17] Robert Amor, Johannes Dimyadi, The promise of automated compliance checking, *Dev. Built Environ.* (ISSN: 26661659) 5 (2021) 100039, <http://dx.doi.org/10.1016/j.dibe.2020.100039>.
- [18] Tanya Bloch, Rafael Sacks, Comparing machine learning and rule-based inferencing for semantic enrichment of BIM models, *Autom. Constr.* (ISSN: 09265805) 91 (2018) 256–272, <http://dx.doi.org/10.1016/j.autcon.2018.03.018>.
- [19] Ling Ma, Rafael Sacks, Uri Kattel, Tanya Bloch, 3D object classification using geometric features and pairwise relationships, *Comput.- Aided Civ. Infrastruct. Eng.* (ISSN: 1093-9687) 33 (2) (2018) 152–164, <http://dx.doi.org/10.1111/mice.12336>.
- [20] Duygu Utkucu, Rafael Sacks, MEP domain object classification through interdomain rule-based semantic enrichment on knowledge graphs, in: 2023 European Conference on Computing in Construction 40th International CIB W78 Conference, Heraklion, Greece, 2023, [https://ec-3.org/publications/conferences/EC32023/papers/EC32023\\_{ }165.pdf](https://ec-3.org/publications/conferences/EC32023/papers/EC32023_{ }165.pdf). (Accessed 01 June 2024).
- [21] Timur Weilbach-Eyüboğlu, Cornelius Preidel, Simon Vilgertshofer, From geometry to semantics: Elevating building model semantics with geometric intelligence, in: *Proceedings of the 41st International Conference of CIB W78, Marrakech*, (ISSN: 2706-6568) 2024, URL <https://itc.scix.net/paper/w78-2024-50>. (Accessed 01 June 2024).
- [22] Michael P. Gallaher, Alan C. O'Connor, John L. Dettbarn Jr., Linda T. Gilday, Cost Analysis of Inadequate Interoperability in the U.S. Capital Facilities Industry, Technical Report, U.S. Department of Commerce Technology Administration, 2004, <http://dx.doi.org/10.6028/NIST.GCR.04-867>, URL <https://nvlpubs.nist.gov/nistpubs/gcr/2004/NIST.GCR.04-867.pdf>. (Accessed 20 August 2024).
- [23] Fábio Matoso Dinis, João Poças Martins, Ana Sofia Guimarães, Bárbara Rangel, BIM and semantic enrichment methods and applications: A review of recent developments, *Arch. Comput. Methods Eng.* (ISSN: 1134-3060) 29 (2) (2022) 879–895, <http://dx.doi.org/10.1007/s11831-021-09595-6>.
- [24] Thomas Krijnen, Martin Tamke, Assessing implicit knowledge in BIM models with machine learning, *Model. Behav.* (2015) 397–406, [http://dx.doi.org/10.1007/978-3-319-24208-8\\_{ }33](http://dx.doi.org/10.1007/978-3-319-24208-8_{ }33).
- [25] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, Justin M. Solomon, Dynamic graph CNN for learning on point clouds, *ACM Trans. Graph.* 38 (2019) [arXiv:1801.07829](https://arxiv.org/abs/1801.07829).
- [26] Christoph Emunds, Nicolas Pauen, Veronika Richter, Jérôme Frisch, Christoph van Treeck, Sparse-BIM: Classification of IFC-based geometry via sparse convolutional neural networks, *Adv. Eng. Inform.* (ISSN: 1474-0346) 53 (2022) 101641, <http://dx.doi.org/10.1016/j.aei.2022.101641>.
- [27] Yutong Feng, Yifan Feng, Haoxuan You, Xibin Zhao, Yue Gao, Meshnet: Mesh neural network for 3d shape representation, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33, 2019, pp. 8279–8286.
- [28] Charles Qi, Su Hao, Kaichun Mo, Guibas Leonidas, PointNet: Deep learning on point sets for 3D classification and segmentation, in: *IEEE Conference on Computer Vision and Pattern Recognition (Ed.)*, 30th IEEE Conference on Computer Vision and Pattern Recognition, IEEE, Piscataway, NJ, ISBN: 978-1-5386-0457-1, 2017, pp. 77–85, <http://dx.doi.org/10.1109/CVPR.2017.16>.
- [29] Johan Luttun, Thomas Krijnen, IfcVoxNet: 3D segmentation and classification of voxelized IFC building models with deep learning, in: *Proceedings of the 41st International Conference of CIB W78, Marrakech*, (ISSN: 2706-6568) 2024, URL <https://itc.scix.net/paper/w78-2024-68>. (Accessed 01 June 2024).
- [30] Marina Petrochenko, Pavel Nedviga, Anna Kukina, Kseniya Strelets, Valeriya Sherstyuk, Machine learning model for the BIM classification in IFC format, *Mag. Civ. Eng.* (ISSN: 2712-8172) (2024) <http://dx.doi.org/10.34910/MCE.126.2>.
- [31] Ruoming Zhai, Jingui Zou, Yifeng He, Liyuan Meng, BIM-driven data augmentation method for semantic segmentation in superpoint-based deep learning network, *Autom. Constr.* (ISSN: 0926-5805) 140 (2022) 104373, <http://dx.doi.org/10.1016/j.autcon.2022.104373>.
- [32] Bonsang Koo, Sunmin La, Nam-Wook Cho, Youngsu Yu, Using support vector machines to classify building elements for checking the semantic integrity of building information models, *Autom. Constr.* (ISSN: 0926-5805) 98 (2019) 183–194, <http://dx.doi.org/10.1016/j.autcon.2018.11.015>.
- [33] Kay Rogage, Omar Doukari, 3D object recognition using deep learning for automatically generating semantic BIM data, *Autom. Constr.* (ISSN: 0926-5805) 162 (2024) 105366, <http://dx.doi.org/10.1016/j.autcon.2024.105366>.
- [34] Fiona Collins, Alexander Braun, Martin Ringsquandl, Daniel M. Hall, Andre Borrmann, Assessing IFC classes with means of geometric deep learning on different graph encodings, in: *Proceedings of the 2021 European Conference on Computing in Construction*, in: *Computing in Construction*, vol. 2, European Council on Computing in Construction, Online Conference, (ISSN: 2684-1150) ISBN: 978-3-907234-54-9, 2021, pp. 332–341, <http://dx.doi.org/10.35490/EC3.2021.168>.
- [35] Zijian Wang, Rafael Sacks, Timson Yeung, Exploring graph neural networks for semantic enrichment: Room type classification, *Autom. Constr.* 134 (2022) <http://dx.doi.org/10.1016/j.autcon.2021.104039>.
- [36] Adam Buruzs, Miloš Šipetić, Brigitte Blank-Landeshammer, Gerhard Zucker, IFC BIM model enrichment with space function information using graph neural networks, *Energies* (ISSN: 1996-1073) 15 (8) (2022) <http://dx.doi.org/10.3390/en15082937>.
- [37] Guy Austern, Tanya Bloch, Yael Abulafia, Incorporating context into BIM-derived data—Leveraging graph neural networks for building element classification, *Buildings* (ISSN: 2075-5309) 14 (2) (2024) <http://dx.doi.org/10.3390/buildings14020527>, URL <https://www.mdpi.com/2075-5309/14/2/527>.
- [38] Artur Tomczak, Wojciech Teclaw, Marcin Luczkowski, The geometry of 884k construction products extracted from IFC/BIM models with IFC labels, 2024, <http://dx.doi.org/10.5281/zenodo.10730758>.
- [39] Christoph Emunds, Nicolas Pauen, Veronika Richter, Jérôme Frisch, Christoph van Treeck, Ifcnet: A benchmark dataset for IFC entity classification, in: *Proceedings of the 28th International Workshop on Intelligent Computing in Engineering (EG-ICE)*, 2021, <http://dx.doi.org/10.48550/arXiv.2106.09712>.
- [40] The University of Auckland School of Computer Science, Openifc model repository, 2021, URL <https://openifcmodel.cs.auckland.ac.nz/>. (Accessed 21 July 2024).

- [41] buildingSMART International, Sample & test files, 2021, URL <https://github.com/buildingSMART/Sample-Test-Files>. (Accessed 01 June 2024).
- [42] Nicolas Pauen, Lara Unruh, Dominik Schlütter, Jaroslaw Siwiecki, Jérôme Frisch, Christoph van Treeck, Technical Report: IFC Modell DigitalHub, Technical Report, E3D – Institute of Energy Efficiency and Sustainable Building, RWTH Aachen University, Germany, 2020, URL <https://publications.rwth-aachen.de/record/807912/files/807912.pdf>. (Accessed 13 February 2024).
- [43] Andre Wisen, BIM whale sample IFC files, 2020, URL <https://github.com/andrewisen/bim-whale-ifc-samples>. (Accessed 01 June 2024).
- [44] National Institute of Building Sciences (NIBS), WBDG - whole building design guide, 2020, URL [www.wbdg.org/](http://www.wbdg.org/). (Accessed 01 June 2024).
- [45] Construction Planen-bauen 4.0 Society for the Digitalization of Planning, Operation mbH, IFC für den infrastrukturbereich, 2020, URL <https://ifcinfra.de/>. (Accessed 01 June 2024).
- [46] OSArch, AECO workflow examples, 2021, URL [https://wiki.osarch.org/index.php?title=AECO\\_Workflow\\_Examples](https://wiki.osarch.org/index.php?title=AECO_Workflow_Examples). (Accessed 03 July 2024).
- [47] Max Jaderberg, Karen Simonyan, Andrew Zisserman, Koray Kavukcuoglu, Spatial transformer networks, in: *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, NIPS '15, MIT Press, Cambridge, MA, USA, 2015, pp. 2017–2025.
- [48] Thomas Krijnen, Dion Moullet, IfcOpenShell community, Ifcopenshell: Open source IFC library and geometry engine, 2025, URL <https://ifcopenshell.org/>, Software, version 0.8.1, accessed: 2025-02-26, Online.
- [49] Michael Dawson-Haggerty, Trimesh, 2019, <https://trimesh.org/>, Version 3.2.0, Accessed: 2024-06-01.